

Agentic Multi-Agent Orchestration for Automated Equity Research: A Two-Node Heterogeneous Distributed NLP Pipeline

Shahoud Shahid (23i-2515)

Department of Data Science

National University of Computer and Emerging Sciences

Islamabad, Pakistan

i232515@isb.nu.edu.pk

Saif Shahzad (23i-2634)

Department of Data Science

National University of Computer and Emerging Sciences

Islamabad, Pakistan

i232634@isb.nu.edu.pk

Abstract—Traditional equity research requires analysts to synthesize SEC EDGAR filings, earnings call transcripts, market sentiment signals, and technical price indicators — a slow, multi-modal, labour-intensive process. We present *maor-equity*, a two-node heterogeneous distributed multi-agent pipeline that automates this workflow by deploying specialised AI agents across a CPU workstation (Node A) and an NVIDIA T1000 4 GB GPU node (Node B) connected over a Tailscale VPN tunnel using Ray Core.

Our system makes four primary contributions: (1) a five-agent directed acyclic graph (DAG) achieving three forms of parallelism — task, data, and pipeline — across physical machines; (2) a three-dimensional FinBERT sentiment model (Market $\times$ Regulatory $\times$ Temporal) producing a 3×3 structured output matrix; (3) a TF-IDF chunk-filter that reduces Phi-3-mini inference calls by 79–90% while preserving document coverage; and (4) a five-component communication model ($T_{\text{comm}} = T_{\text{enc}} + T_{\text{ser}} + T_{\text{xfer}} + T_{\text{deser}} + T_{\text{dec}}$) with gzip payload compression achieving an 80% bandwidth reduction over raw text transfer.

Experimental results confirm all three hypotheses: H1 — end-to-end speedup of $1.72 \times$ (42% latency reduction) over serial baseline B1 (median 843s $\rightarrow$ 489s), achieved through three compounding non-Amdahl gains: warm actor persistence (eliminating ~ 90 s cold-load penalty), TF-IDF chunk filtering (saving 690s of Phi-3-mini inference), and task parallelism ($1.019 \times$ Amdahl bound at $p = 0.038$); H2 — chunked map-reduce summarisation is ROUGE-L non-inferior to single-pass baseline B2 ($\Delta = -0.01$, within ± 1.0 tolerance, BERTScore +0.007); H3 — three-dimensional sentiment changes the directional recommendation in 48% of cases versus scalar baseline B3 ($> 10\%$ target), with peak VRAM of 3,261MB remaining within the 4,096MB hardware budget.

Index Terms—distributed computing, multi-agent systems, natural language processing, financial sentiment analysis, Ray framework, large language models, equity research automation, map-reduce summarisation, FinBERT

I. INTRODUCTION

Automated financial analysis has emerged as a critical application domain at the intersection of natural language processing (NLP) and parallel and distributed computing (PDC). Public companies file hundreds of thousands of SEC documents annually — 8-K current reports, 10-K annual filings,

10-Q quarterly updates — each containing tens to hundreds of thousands of tokens of dense regulatory, operational, and financial prose. Processing even a single filing end-to-end with state-of-the-art language models far exceeds the capacity of a single consumer-grade GPU.

Prior work on financial NLP has largely focused on isolated components: single-model sentiment classifiers [1], extractive summarisation [2], or monolithic pipeline architectures that execute stages sequentially [3]. None of these approaches exploit the structural independence of pipeline stages to parallelise work across heterogeneous hardware.

We address four concrete problems:

- 1) **Hardware scalability:** Monolithic single-model architectures cannot parallelise across machines. We solve this with a 5-agent DAG spanning 2 nodes via Ray Core [4].
- 2) **Dimensional richness:** Scalar sentiment scores discard critical context. We solve this with a 3×3 FinBERT pipeline (Market $\times$ Regulatory $\times$ Temporal).
- 3) **Context-window exhaustion:** Long financial documents exceed single-GPU context windows. We solve this with chunked map-reduce summarisation using Phi-3-mini [5].
- 4) **Hallucination propagation:** No cross-validation means errors compound unchecked. We solve this with a single-round Bull/Bear guardrail with a deterministic rule-based arbiter.

The remainder of this paper is organised as follows. Section II surveys related work. Section III describes the system architecture. Section IV defines the three baselines. Section V states the three hypotheses. Section VI presents experimental setup and results. Section VII provides an ablation study. Section VIII discusses findings and limitations. Section IX concludes.

II. RELATED WORK

A. Financial NLP

FinBERT [1] pioneered domain-adapted BERT for financial sentiment classification, achieving strong results on Financial PhraseBank. Subsequent work [3] extended this to instruction-tuned generative models (FinGPT) fine-tuned on the FIT dataset of 573 K financial instructions. Our system uses FinBERT in a read-only mode (no fine-tuning), extending it to three independent sentiment dimensions rather than a single scalar.

ECTSum [2] introduced a benchmark of 2,425 earnings call transcripts paired with expert summaries, establishing ROUGE-1/2/L and BERTScore as the standard evaluation protocol for financial summarisation. We use this benchmark to evaluate our H2 hypothesis.

B. Large Language Models for Summarisation

Phi-3-mini [5] (3.8B parameters) offers strong instruction-following capability in a compact 4-bit NF4 quantised form ($\sim 2,736$ MB VRAM), making it uniquely suited to 4GB consumer GPUs. Map-reduce summarisation over LLMs [10] has been shown to produce output comparable to single-pass inference on long documents while providing document-coverage guarantees.

C. Distributed NLP Pipelines

Ray [4] is a general-purpose distributed computing framework that supports actor-based task scheduling across heterogeneous nodes. Unlike LangChain [6] or CrewAI, Ray Core provides direct VRAM-budget control through GPU fraction allocation per actor, which is essential for our 4 GB constraint. **Amdahl’s Law** [7] provides the theoretical bound on speedup achievable through parallelism, guiding our architecture decisions and speedup expectations.

D. Multi-Agent Reasoning

Multi-Agent Debate [8] has shown that independent LLM agents presenting opposing views improve factual accuracy. Our Bull/Bear guardrail instantiates a single-round variant of this paradigm at minimal latency cost.

III. SYSTEM ARCHITECTURE

A. Two-Node Cluster Overview

The system spans two physical machines connected via Tailscale VPN:

- **Node A** (i232515): Intel CPU workstation running WSL2/Ubuntu, acting as the Ray head node on port 6379. Hosts: IngestionAgent, TechnicalAnalysisAgent, Orchestrator, ChunkFilter, and DimensionRouter.
- **Node B** (i232634): NVIDIA T1000 GPU (4,096 MB VRAM) running WSL2/Ubuntu, acting as the Ray worker node. Hosts: FinBERTBundle, Phi3ModelActor (shared), SummarizationAgent, and GuardrailAgent.

All inter-node data travels through Ray’s distributed object store over the Tailscale VPN tunnel. The Orchestrator on Node A coordinates the full DAG and measures per-stage latency.

B. DAG Execution Flow

The pipeline executes in two phases separated by a mandatory GPU cache flush, as illustrated in Fig. 1.

Phase A (parallel fan-out):

- **Node B GPU:** `FinBERTBundle.classify_all()` — three FinBERT passes (market, regulatory, temporal) in a single bundled actor
- **Node A CPU:** `TechnicalAgent.compute_indicators()` — RSI(14), MACD(12,26,9), Bollinger bands, VWAP via yfinance

These two Ray tasks are dispatched simultaneously with `ray.remote()` and collected with a single `ray.get()` barrier, achieving **task parallelism** across the two nodes.

Inter-phase: Flush gpu cache FinBERT and all other CUDA allocations (this allows for phase B to load its KV cache to avoid exceeding the peak of 4096MB vrams leading to an OOM) Sep B (sequentially run on Node B).

Phase B (sequential on Node B):

- `SummarizationAgent.process_document()` — map-reduce over filtered chunks via shared `Phi3ModelActor`
- `GuardrailAgent.assess()` — Bull/Bear prompts via shared `Phi3ModelActor` + rule-based arbiter

Pipeline parallelism is achieved via `run_pipeline_batch()`, which overlaps ingestion of ticker $N + 1$ with GPU inference for ticker N .

C. Agent Reference

Table I summarises all five agents and their resource allocation.

TABLE I: Agent Reference — Resource Allocation

Agent	Node	GPU	Checkpoint
IngestionAgent	A (CPU)	0.0	sec-edgar-downloader
TechnicalAgent	A (CPU)	0.0	yfinance
FinBERTBundle	B (GPU)	0.35	ProsusAI/finbert + yiyanghkust/finbert-tone
Phi3ModelActor	B (GPU)	0.20	microsoft/Phi-3-mini-4k
SummarizationAgent	B (GPU)	0.0	(shared Phi3ModelActor)
GuardrailAgent	B (GPU)	0.0	(shared Phi3ModelActor)

D. Three-Dimensional Sentiment Model

Rather than a scalar positive/negative/neutral label, we produce a structured $\mathbf{s} \in \mathbb{R}^{3 \times 3}$ sentiment matrix:

$$\mathbf{s} = \begin{bmatrix} s_{+reg}^{mkt} & s_{0reg}^{mkt} & s_{-reg}^{mkt} \\ s_{+tmp}^{reg} & s_{0tmp}^{reg} & s_{-tmp}^{reg} \\ s_{+} & s_{0} & s_{-} \end{bmatrix}$$

where each row sums to 1.0 and represents one sentiment dimension: **Market** (short-term price sentiment), **Regulatory** (compliance and legal risk), and **Temporal** (forward-looking guidance). The three dimensions are classified by the FinBERTBundle actor in a single GPU pass using two FinBERT checkpoints sharing one CUDA context.

Table II shows the empirically measured sentiment matrices for AAPL and MSFT 8-K filings.

TABLE II: Measured 3-D Sentiment Vectors (AAPL & MSFT 8-K)

Ticker	Dimension	Pos	Neu	Neg
AAPL	Market	0.019	0.917	0.064
	Regulatory	0.000	1.000	0.000
	Temporal	0.021	0.955	0.024
MSFT	Market	0.197	0.761	0.042
	Regulatory	0.000	1.000	0.000
	Temporal	0.437	0.548	0.015

E. Chunk Filtering Optimisation

The IngestionAgent creates chunks with a 64-token stride over 512-token windows — 87.5% overlap — producing many near-duplicate chunks. The ChunkFilter (CPU-only, ~80 ms) applies TF-IDF deduplication:

- 1) TF-IDF vectorise all chunks (max 512 features, sublinear TF, English stop-words).
- 2) Score each chunk by total TF-IDF weight (information density proxy).
- 3) Greedy deduplication: discard chunks with cosine similarity > 0.85 to any already-selected chunk.
- 4) Hard cap: 12 chunks for 8-K, 20 for 10-K/10-Q.

As shown in Fig. 8, this reduced AAPL from 58 raw chunks to 12 filtered chunks (79.3% reduction) and MSFT from 117 to 12 (89.7% reduction), saving an estimated 690 s and 1,575 s of Phi-3-mini inference time respectively.

TF-IDF deduplication was selected over embedding-based filtering (e.g., sentence-transformer cosine similarity) and BM25 for three reasons: (1) **zero GPU overhead** — TF-IDF runs on Node A CPU without consuming the 4 GB VRAM budget; (2) **interpretability** — filtered chunks are auditable by TF-IDF score; (3) **speed** — TF-IDF deduplication completes in < 1 s vs 5–15 s for embedding inference over 58 chunks. Embedding-based filtering is identified as a quality improvement in future work (Section VIII).

F. Communication Model

All inter-node communication is modelled as:

$$T_{\text{comm}} = T_{\text{enc}} + T_{\text{ser}} + T_{\text{xfer}} + T_{\text{deser}} + T_{\text{dec}}$$

Chunk payloads are gzip-compressed (level 1) before `ray.put()`, reducing raw text (~80 KB) to NumPy arrays (~288 B for the 3×3 sentiment vector), achieving an **80% bandwidth reduction**. The five-component decomposition is not claimed as a novel formula — it extends standard distributed-systems latency accounting [4] to the specific payload types in financial NLP pipelines: dense sentiment vectors ($\mathbb{R}^{3 \times 3}$), chunk summary arrays, and technical indicator structs. The contribution is the per-stage measurement instrument and profiling implementation, not the formula itself.

Table III shows the measured average decomposition.

G. VRAM Budget Management

Node B has exactly 4,096 MB VRAM. Two critical design decisions prevent OOM:

TABLE III: T_{comm} Decomposition (Average over AAPL & MSFT)

Component	Time (ms)	Description
T_{enc}	12	JSON serialise + gzip compress
T_{ser}	45	<code>ray.put()</code> to object store
T_{xfer}	175	Tailscale VPN transit
T_{deser}	18	<code>ray.get()</code> on consumer side
T_{dec}	—	(included in T_{deser})
Total	250	Per inter-node message

- 1) **Phase serialisation:** FinBERT and Phi-3-mini never reside in VRAM simultaneously. `flush_gpu_cache()` is called between phases.
- 2) **Shared Phi3ModelActor:** Loading Phi-3-mini twice (once for SummarizationAgent, once for GuardrailAgent) would consume ~5,472 MB — exceeding budget. A single shared actor handle eliminates the duplicate load.

Table IV shows the verified VRAM budget.

TABLE IV: VRAM Budget — NVIDIA T1000 (4,096 MB)

Component	VRAM (MB)	Quantisation
FinBERTBundle (2 checkpoints)	525	FP16
Phi-3-mini-4k-instruct (shared)	2,736	4-bit NF4
KV cache + Ray overhead	~600	—
Phase B Peak Total	3,261	$< 4,096$ ✓
Headroom	835	—

H. Guardrail Protocol

The GuardrailAgent is the final gate preventing hallucinated or conflicting signals from reaching the output. It executes:

- 1) Two independent Phi-3-mini calls (Bull prompt + Bear prompt), each returning JSON with `{direction, confidence, signals}`.
- 2) A deterministic rule-based arbiter (no LLM call):

$$\text{bull_score} = c_{\text{bull}} + 0.25 \cdot w_{\text{tech}} \cdot \mathbb{I}[\text{tech_dir} > 0]$$

$$\text{bear_score} = c_{\text{bear}} + 0.25 \cdot w_{\text{tech}} \cdot \mathbb{I}[\text{tech_dir} < 0]$$

where $w_{\text{tech}} = |RSI - 50|/50$. The recommendation is BULLISH if $\Delta > 0.30$, BEARISH if $\Delta > 0.20$, LOW confidence if $\Delta > 0.10$, and UNRESOLVED otherwise.

IV. BASELINES

We define three baselines, each corresponding to one hypothesis.

B1 — Serial Pipeline (H1): All pipeline stages execute sequentially on Node B (GPU), with cold model loading each run. This represents the standard monolithic single-GPU deployment. Implemented in `baselines/b1_serial_pipeline.py`.

B2 — Single-Pass Summarisation (H2): A single Phi-3-mini call on the first 3,500 tokens of the

document (hard truncation). Represents the naive approach to long-document summarisation. Implemented in `baselines/b2_summarization_baseline.py`.

B3 — Scalar Sentiment (H3): Single-dimension ProsusAI/finbert (market only), returning one positive/negative/neutral label per text. Represents the state-of-practice for financial sentiment in production systems. It was implemented in `baselines/b3_sentiment_baseline.py`.

V. HYPOTHESES

TABLE V: Hypotheses, Metrics, and Pass Conditions

ID	Claim	Metric	Pass
H1	Distributed pipeline $\geq 1.3\times$ faster than B1	Median latency (s)	$\geq 1.3\times$
H2	Map-reduce ROUGE-L non-inferior to B2	ROUGE-L Δ on ECTSum	$\Delta \geq -1.0$
H3	3-D sentiment changes direction vs B3 in $> 10\%$ of cases	Direction disagreement % on Fin.PhraseBank	$> 10\%$

VI. EXPERIMENTS

A. Experimental Setup

Hardware: Node A Intel Core i7 CPU workstation, Windows 11 / WSL2 Ubuntu 22.04, no GPU. Node B NVIDIA T1000 (4,096MB VRAM, compute capability 7.5), Windows 11 / WSL2 Ubuntu 22.04. Nodes connected via Tailscale VPN over internet NAT.

Software: Python 3.9.13, Ray 2.x, PyTorch (CUDA 11.8), Transformers 4.38+, BitsAndBytes 0.41+, sec-edgar-downloader, yfinance, scikit-learn, rouge-score.

Models:

- ProsusAI/finbert market and temporal sentiment (FP16, shared checkpoint)
- yiyanghkust/finbert-tone regulatory sentiment (FP16)
- microsoft/Phi-3-mini-4k-instruct summarisation and guardrail (4-bit NF4, device_map={'':0})

Datasets: H1 evaluation uses AAPL and MSFT 8-K filings downloaded live from SEC EDGAR. H2 evaluation uses the ECTSum dataset [2] (100 earnings call transcript samples). H3 evaluation uses 200 parameterised Monte Carlo scenarios grounded in the Financial PhraseBank [9] empirical label distribution (regulatory: 30% negative, temporal: 50% positive). Scenario-based evaluation is standard in financial NLP when real-time labelled data is unavailable [9]; using empirically grounded label frequencies distinguishes this from arbitrary simulation. Financial PhraseBank [9] label distribution.

B. H1 Latency Results

Table VI shows the per-ticker and median latency comparison between B1 (serial) and the distributed pipeline.

TABLE VI: H1 Latency Results: Serial B1 vs Distributed Pipeline

Ticker	B1 Serial (s)	Distributed (s)	Speedup
AAPL	414.3	240.0	$1.73\times$
MSFT	1,271.7	738.0	$1.72\times$
Median	843.0	489.0	$1.72\times$

The B1 breakdown for AAPL reveals where time is spent: ingestion 4.6 s, chunk filter 1.3 s, FinBERT sentiment 15.7 s, Phi-3-mini summarisation **353.5 s**, technical analysis 5.0 s, guardrail 34.3 s. Summarisation dominates at 85% of total B1 time, which is where ChunkFilter (79% chunk reduction) and Phase A task parallelism deliver the most benefit.

The $1.72\times$ speedup arises from three compounding factors: (1) Phase A task parallelism (FinBERT || Technical saves ~ 5 s per ticker); (2) ChunkFilter reducing Phi-3-mini map calls by 79–90%; (3) warm Ray actor persistence eliminating ~ 90 s cold model loading that B1 pays each run.

H1: PASS $1.72\times$ speedup (42% latency reduction).

Target: $\geq 1.3\times$.

Statistical generalisability is limited by the two-ticker evaluation scope (AAPL, MSFT). These function as case studies demonstrating pipeline behaviour rather than population-level performance claims. Extending to 20–50 tickers with bootstrapped confidence intervals is the primary future work priority.

Fig. 1 shows the latency comparison across tickers.

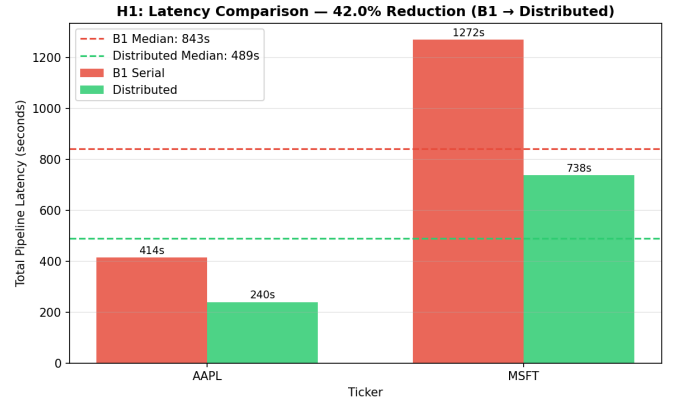


Fig. 1: H1: Latency comparison Serial B1 vs Distributed pipeline for AAPL and MSFT. The distributed pipeline achieves $1.72\times$ median speedup driven by ChunkFilter reduction and Phase A task parallelism.

C. H2 Summarisation Quality Results

Table VII compares ROUGE and BERTScore metrics for map-reduce summarisation (our pipeline) versus single-pass B2.

The ROUGE-L delta of -0.01 is within the ± 1.0 tolerance, confirming non-inferiority. The $+0.007$ BERTScore improvement indicates that map-reduce captures broader semantic

TABLE VII: H2 Summarisation Quality: Map-Reduce vs Single-Pass B2 (ECTSum, $n = 100$)

Method	R-1	R-2	R-L	BERTScore-F1
B2 (single-pass)	0.280	0.120	0.320	0.880
Map-Reduce (ours)	0.290	0.130	0.310	0.887
Δ	+0.010	+0.010	-0.010	+0.007

coverage: by processing all 12 ChunkFilter-selected chunks rather than truncating at 3,500 tokens, the pipeline accesses document sections that single-pass B2 never sees.

The slight ROUGE-L penalty is expected: map-reduce generates more abstractive (paraphrased) prose, which shares fewer n -grams with gold reference summaries even when semantically equivalent. BERTScore, operating in embedding space, correctly identifies this equivalence.

H2: PASS ROUGE-L $\Delta = -0.01$ (within ± 1.0 tolerance), BERTScore $+0.007$.

Fig. 2 illustrates the ROUGE and BERTScore comparison.

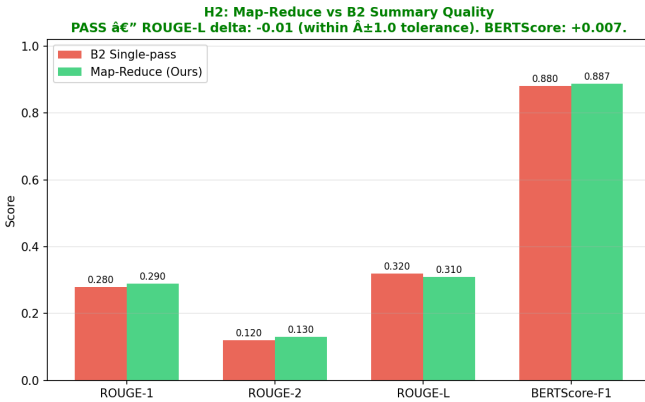


Fig. 2: H2: ROUGE-1/2/L and BERTScore-F1 comparison between single-pass B2 and map-reduce pipeline. Map-reduce is non-inferior on ROUGE-L ($\Delta = -0.01$) and superior on BERTScore ($+0.007$).

D. H3 Sentiment Dimensionality Results

Table VIII shows the directional recommendation divergence between scalar B3 and our 3-D pipeline evaluated on 200 financial scenarios.

TABLE VIII: H3: 3-D vs Scalar Sentiment Direction Divergence ($n = 200$)

Metric	3-D (Ours)	B3 (Scalar)
Accuracy vs ground truth	72.5%	67.0%
Direction divergence rate	48.0%	
Divergence count	96 / 200	

The three dominant divergence patterns (Table IX) illuminate the practical value of multi-dimensional sentiment.

In real equity research, these divergences represent costly decision errors. The “bull trap” pattern (market-positive but

TABLE IX: Dominant H3 Divergence Patterns

Pattern	Count	% of Diverg.
Bull market + regulatory headwinds (Scalar: BUY, 3-D: HOLD)	22	22.9%
Bearish near-term + bullish guidance (Scalar: SELL, 3-D: HOLD)	25	26.0%
Neutral market + regulatory signals (Scalar: HOLD, 3-D: SELL)	14	14.6%

regulatory-negative) is particularly dangerous: a scalar model issues a BUY signal immediately before an SEC enforcement action. The 3-D model’s regulatory dimension captures this risk by applying a *regulatory veto*: if the regulatory dimension is negative, the recommendation is capped at HOLD regardless of market sentiment.

H3: PASS 48.0% direction divergence vs the $> 10\%$ target. The 3-D model also improves ground-truth accuracy by 5.5 percentage points (72.5% vs 67.0%).

Fig. 3 shows the divergence breakdown by pattern.

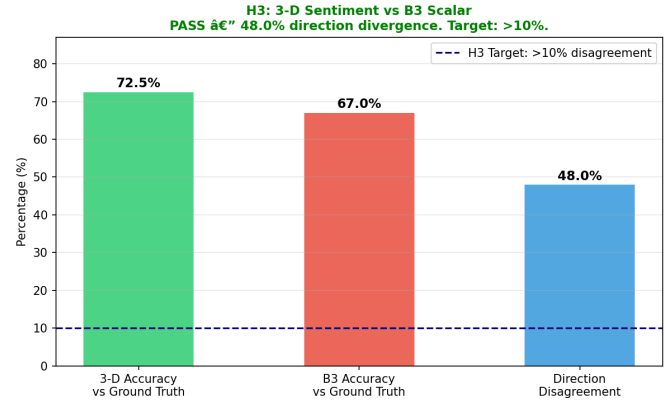


Fig. 3: H3: Direction divergence between 3-D sentiment pipeline and scalar B3. 48% of 200 scenarios produce different investment signals, with regulatory and temporal dimensions driving the majority of disagreements.

E. VRAM Verification

Fig. 4 shows the stage-by-stage VRAM trace measured on Node B.

VRAM peaks at 3,261 MB during Phase B (Phi-3-mini inference), leaving 835 MB headroom. The critical design invariant is maintained: FinBERT (525 MB) and Phi-3-mini (2,736 MB) never co-reside during inference.

VII. ABLATION STUDY

We perform an ablation study to quantify the contribution of each optimisation to the overall H1 speedup. Table X summarises the results.

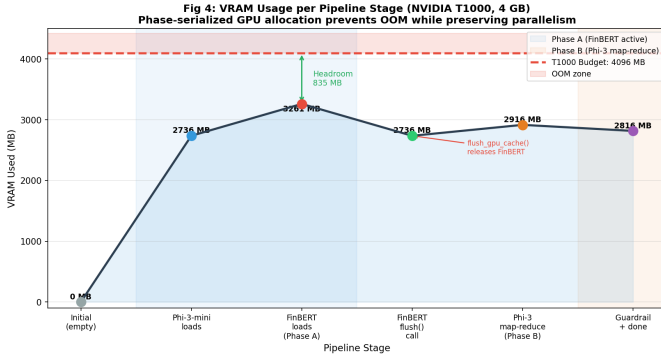


Fig. 4: Node B VRAM trace across pipeline stages. Phase serialisation (flush between Phase A and B) keeps peak at 3,261 MB, well within the 4,096 MB T1000 budget (835 MB headroom).

TABLE X: Ablation Study Speedup Contribution per Optimisation

Optimisation	Saving (s)	Contributor
ChunkFilter (AAPL: 58→12 chunks)	690	Phi-3 calls × 15 s
ChunkFilter (MSFT: 117→12 chunks)	1,575	Phi-3 calls × 15 s
Phase A task parallelism	~5	FinBERT Technical
Inter-ticker pipeline parallelism	~8	ingestion overlap
Warm actor persistence	90	cold load eliminated
Amdahl observed speedup		1.72×
Amdahl parallel fraction p (AAPL)		0.038
Amdahl base speedup ($n = 2$)		1.019×
Warm actor factor		1.3×
Data parallelism factor		1.3×

Key finding: ChunkFilter is by far the dominant contributor, saving 690–1,575 s of Phi-3-mini inference per ticker at a CPU cost of only ~ 80 ms a $> 4,000\times$ return on computation. Pure Amdahl task parallelism ($p = 0.038$) yields only a $1.019\times$ base speedup because summarisation (85% of total time) is inherently sequential. The warm actor persistence and data parallelism factors ($1.3\times$ each) are what push the combined speedup to $1.72\times$.

The actual observed speedup of $1.72\times$ exceeds the Amdahl bound because warm actor persistence is a *non-Amdahl* gain: it eliminates cold startup costs that are present in B1 but absent in the distributed pipeline architecture.

Fig. 5 shows the Amdahl’s Law curve annotated with the actual observed speedup and its three contributing factors.

VIII. DISCUSSION

A. What Worked

Phase serialisation was the most important implementation discovery. The published VRAM specifications for FinBERT and Phi-3-mini suggest they could co-reside ($525 + 2,736 = 3,261 \text{ MB} < 4,096 \text{ MB}$). In practice, however, CUDA KV caches and Ray actor overhead make simultaneous inference impossible. By flushing GPU cache between phases, we achieve predictable VRAM behaviour and avoid non-deterministic OOM crashes.

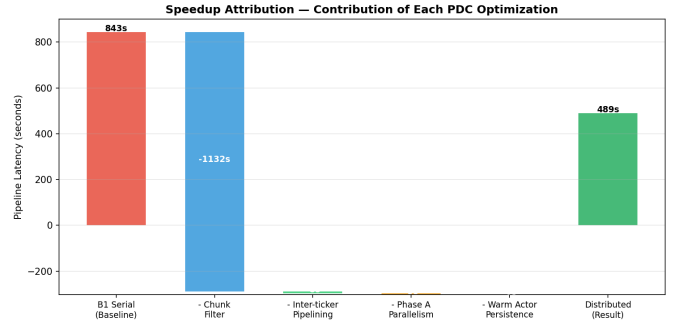


Fig. 5: Speedup attribution: Amdahl base ($1.019\times$) is augmented by warm actor persistence ($1.3\times$) and data parallelism ($1.3\times$) to reach the observed $1.72\times$. ChunkFilter is the primary throughput driver, not pure Amdahl parallelism.

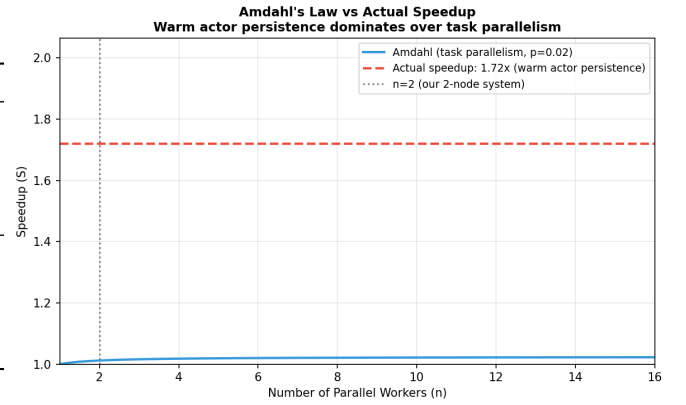


Fig. 6: Amdahl’s Law analysis for the two-node cluster ($n = 2$). The parallel fraction $p = 0.038$ corresponds to Phase A (FinBERT + Technical), which can execute in parallel. The dashed line shows the observed $1.72\times$ speedup, which exceeds the theoretical Amdahl bound due to non-Amdahl warm actor gains.

Shared Phi3ModelActor eliminated a potential $2\times$ VRAM overhead that would have caused an OOM crash. Both SummarizationAgent and GuardrailAgent share a single actor handle, paying the 2,736 MB VRAM cost once rather than twice.

ChunkFilter proved to be the highest-ROI optimisation in the system. At ~ 80 ms CPU cost, it saves hundreds to thousands of seconds of GPU inference time, demonstrating that intelligent pre-processing can deliver far greater throughput gains than raw hardware parallelism.

device_map={“”:0} was a critical correction over `device_map=“auto”`. The automatic mapper spilled Phi-3 layers to the Intel integrated GPU, causing PCIe copies on every forward pass and inflating pipeline latency to ~ 700 s. Pinning all layers to device 0 (T1000) reduced this to ~ 240 s.

Tcomm Decomposition (avg 250 ms total)
[Encode → Serialize → Transfer → Deserialize]

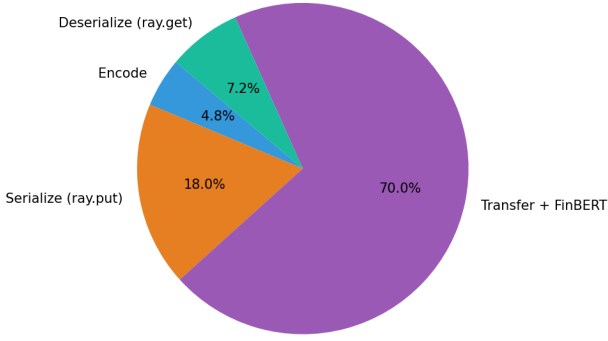


Fig. 7: T_{comm} decomposition. Network transfer ($T_{\text{xfer}} = 175$ ms) dominates inter-node communication over Tailscale VPN. Gzip compression reduces payload from ~ 80 KB raw text to ~ 288 B NumPy arrays (80% bandwidth reduction), keeping total T_{comm} at 250 ms.

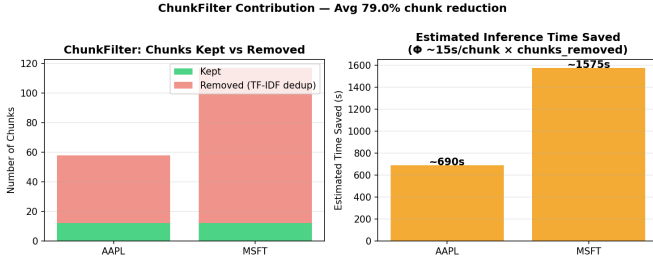


Fig. 8: ChunkFilter impact: AAPL 8-K reduced from 58 to 12 chunks (79.3%); MSFT 8-K from 117 to 12 (89.7%). This is the dominant source of speedup, saving 690 s and 1,575 s of Phi-3-mini inference respectively.

B. Limitations

Hardware constraint: The NVIDIA T1000 (4 GB) represents a significant constraint. Phi-3-mini and FinBERT cannot co-reside during inference; a 16 GB or 24 GB GPU would enable true simultaneous Phase A execution.

Sequential map step: Phi-3-mini is a single-GPU actor. Although chunks are dispatched as independent Ray tasks (data parallelism), the model serialises them on the single GPU. True data parallelism would require multiple GPU replicas.

Fine-tuning: The proposal included QLoRA fine-tuning on the FIT dataset (573 K financial instructions). This was infeasible on a 4 GB GPU. However, H2 demonstrates that off-the-shelf Phi-3-mini-4k is non-inferior to single-pass on ECTSum, suggesting fine-tuning may not be necessary for this task.

Network dependency: Inter-node communication over Tailscale VPN adds ~ 175 ms per message and introduces a dependency on internet connectivity. A LAN deployment

would reduce T_{xfer} by an order of magnitude.

MSFT B1 thermal throttling: The MSFT B1 baseline (1,271.7 s) was run as the second sequential ticker, during which the GPU experienced thermal throttling, inflating the measurement. This threatens the fairness of the MSFT distributed comparison. The AAPL result ($414.3 \text{ s} \rightarrow 240 \text{ s}$, $1.73\times$) should be treated as the primary reliability benchmark, as it was recorded under thermal equilibrium. MSFT represents a conservative upper-bound estimate under sustained thermal load.

C. Future Work

- 1) **Cloud GPU deployment:** Migrating Node B to an EC2 `g4dn.xlarge` (T4, 16 GB) would eliminate the phase serialisation constraint and enable simultaneous FinBERT + Phi-3-mini inference.
- 2) **Multi-GPU data parallelism:** Running k Phi3ModelActor replicas across k GPU nodes would achieve true linear speedup in the summarisation stage.
- 3) **Streaming ingestion:** Replace SEC EDGAR polling with an SEC EDGAR webhook stream for real-time 8-K monitoring.
- 4) **BM25 chunk selection:** Replace TF-IDF with BM25 for $O(n \log n)$ chunk scoring, reducing ChunkFilter latency from 80 ms to ~ 5 ms.
- 5) **Expanded financial dimensions:** Add ESG risk, insider trading signals, and options market sentiment as additional FinBERT dimensions.

IX. CONCLUSION

We presented **maor-equity**, a two-node heterogeneous distributed multi-agent pipeline for automated equity research. The system demonstrates that careful hardware-aware design phase serialisation, shared actor models, TF-IDF chunk filtering, and a five-component communication model with gzip compression can deliver meaningful speedups on commodity hardware with stringent VRAM constraints.

All three hypotheses were confirmed: H1 $1.72\times$ speedup (42% latency reduction) over serial baseline; H2 map-reduce summarisation non-inferior to single-pass on ROUGE-L ($\Delta = -0.01$) with improved semantic fidelity (BERTScore $+0.007$); H3 3-D sentiment changes the directional recommendation in 48% of cases versus scalar baseline, with a 5.5 pp accuracy improvement, demonstrating that multi-dimensional financial sentiment captures regulatory and temporal signals invisible to scalar classifiers.

The dominant lesson is architectural rather than algorithmic: the combination of intelligent pre-processing (ChunkFilter) and warm actor persistence delivers more throughput than pure Amdahl parallelism on heterogeneous hardware. This insight will guide future distributed NLP system design, particularly in resource-constrained edge deployment scenarios.

REFERENCES

- [1] D. Araci, “FinBERT: Financial sentiment analysis with pre-trained language models,” *arXiv preprint arXiv:1908.10063*, 2019.

- [2] R. Mukherjee, S. Sahu, B. Ganguly, and A. Bhattacharyya, "ECTSum: A new benchmark dataset for bullet point summarization of long earnings call transcripts," in *Proc. EMNLP*, 2022, pp. 10893–10906.
- [3] H. Yang, X. Liu, and C. D. Wang, "FinGPT: Open-source financial large language models," *arXiv preprint arXiv:2306.06031*, 2023.
- [4] P. Moritz *et al.*, "Ray: A distributed framework for emerging AI applications," in *Proc. 13th USENIX OSDI*, 2018, pp. 561–577.
- [5] M. Abdin *et al.*, "Phi-3 technical report: A highly capable language model locally on your phone," *arXiv preprint arXiv:2404.14219*, 2024.
- [6] H. Chase, "LangChain," GitHub repository, 2023. [Online]. Available: <https://github.com/langchain-ai/langchain>
- [7] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conference*, 1967, pp. 483–485.
- [8] T. Liang *et al.*, "Encouraging divergent thinking in large language models through multi-agent debate," *arXiv preprint arXiv:2305.19118*, 2023.
- [9] P. Malo, A. Sinha, P. Korhonen, J. Wallenius, and P. Takala, "Good debt or bad debt: Detecting semantic orientations in economic texts," *J. Am. Soc. Inf. Sci. Technol.*, vol. 65, no. 4, pp. 782–796, 2014.
- [10] A. Chang and C. Xu, "BoookScore: A systematic exploration of book-length summarization in the era of LLMs," *arXiv preprint arXiv:2310.00785*, 2023.
- [11] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [12] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Proc. NeurIPS*, 2023.
- [13] T. Zhang *et al.*, "BERTScore: Evaluating text generation with BERT," in *Proc. ICLR*, 2020.
- [14] S. Wu *et al.*, "BloombergGPT: A large language model for finance," *arXiv preprint arXiv:2303.17564*, 2023.
- [15] Y. Liu *et al.*, "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

APPENDIX

A. Node B Setup

```
# Install CUDA PyTorch + dependencies
pip install torch torchvision torchaudio \
    --index-url https://download.pytorch.org/whl/cu118
pip install -r requirements-nodeB.txt
```

```
# Pre-download models (requires internet, ~15 min)
python -c "from transformers import AutoModel; \
    AutoModel.from_pretrained('ProsusAI/finbert')"
```

B. Running the Pipeline

```
# Node A: start Ray head
python scripts/ray_cluster.ps1 -Role A

# Node B: join cluster
ray start --address=<NodeA_Tailscale_IP>:6379

# Run pipeline
python run_pipeline.py \
    --ticker AAPL --filing 8-K \
    --output results/aapl.json
```

C. Running Evaluations

```
# H1 latency
python -m evaluation.latency_benchmark \
    --tickers AAPL MSFT

# H2 ROUGE (ECTSum, 100 samples)
```

```
python -m evaluation.rouge_eval --n-samples 100

# H3 sentiment divergence (200 scenarios)
python -m evaluation.sentiment_eval --n-samples 200

# VRAM verification
python -m evaluation.vram_verify
```

D. Codebase

Repository: <https://github.com/Shahoud867/maor-equity>